

SRI SIDDHAROODHA SWAMY TEMPLE DIGITAL MANAGEMENT SYSTEM

In-Depth Technical Documentation & System Reference Guide

1. System Architecture Overview

The application is designed using a decoupled 3-tier architecture, ensuring high performance, scalability, and data integrity. The layers are split into the Presentation Tier (Frontend Client), the Business Logic Tier (REST API Backend Server), and the Data Tier (NoSQL Database).

- **Presentation Tier (React SPA):** Runs inside the user's web browser, managing view logic and user interactions. Communication with the backend is done asynchronously using Axios, which makes it feel fluid without page reloads.
- **Business Logic Tier (Spring Boot Server):** A REST API server written in Java 21 using Spring Boot 3. It parses requests, handles authorization through JWT, controls transactions, validates input parameters, and communicates with external SMTP servers.
- **Data Tier (MongoDB Database):** A document database where data is stored in BSON format, allowing schemas to adjust dynamically based on transaction parameters.

2. Frontend Client Architecture (React + Vite)

The React codebase leverages component modularity and client-side page routing to organize screens cleanly. The main modules include Home (Sun Chakra interactive navigation), Sevas (Gotra-based checkout cart drawer), Prasad (e-commerce catalog), Lodging Rooms (check-in/check-out range calendars), and Admin (data dashboards).

2.1 Package Dependencies Breakdown (package.json)

Package Name	Version	Technical Purpose & Choice
react & react-dom	^19.2.6	Core view library handling virtual DOM manipulation and component state updates.
vite	^8.0.12	Fast module bundler enabling instant hot-reloading (HMR) for developer loops.
react-router-dom	^7.15.1	Allows smooth page routing without reload cycles.
axios	^1.16.1	HTTP client sending JSON payloads and JWT authorization tokens to backend endpoints.
framer-motion	^12.38.0	Controls luxury animations like homepage Sun Chakra rotations.
tailwindcss	^4.3.0	Utility-first styling utility using dynamic variables for a premium gold-saffron color theme.

3. Backend API Server Architecture (Java 21 + Spring Boot 3)

The Spring Boot application handles REST mapping, database connection, secure authentication via Spring Security, and transactional emails. The codebase is compiled using Java 21, leveraging modern language features.

3.1 Backend Dependencies Breakdown (pom.xml)

Dependency artifactId	Framework	Functionality
spring-boot-starter-web	Spring Boot Web	Hosts embedded Tomcat server and provides REST controllers mapping.
spring-boot-starter-data-mongodb	Spring Data	Manages MongoDB driver integrations and MongoRepository mappings.
spring-boot-starter-security	Spring Security	Secures REST APIs, handles password hashing, and CORS filtering.
jjwt-api, jjwt-impl	JJWT Library	Constructs, signs, and parses JSON Web Tokens for secure seeker sessions.
spring-boot-starter-mail	JavaMail SMTP	Integrates SMTP client sending rich-text HTML receipt templates.

4. Database Architecture & Collections (MongoDB)

MongoDB acts as the persistent document store. Instead of SQL tables, documents are stored in collections in BSON format. This allows gotras, room counts, and donation fields to remain flexible.

4.1 Core Document Schemas

- **users Collection:** Stores registered devotee credentials.
- **bookings Collection:** Logs Sevas bookings (Gotra, Nakshatra, booking date, amount paid).
- **room_bookings Collection:** Tracks lodging slots (check-in, check-out dates, status, total cost).
- **donations Collection:** Records charity contributions (purpose, email, amount, transaction hash).
- **darshan_bookings Collection:** Records crowd control entry passes (scheduled date, time slot, group count, and ticket scan status).

5. Real-World Feature Addition: Smart Darshan Queue Booking

To address festival overcrowding (e.g. Mahashivratri) and stampede hazards, we implemented a digital booking slot gate pass. Devotees select their scheduled visiting hour online, and the system verifies if there is available hourly capacity before saving to MongoDB. If successful, the devotee receives a confirmation email with a scan-verifiable QR code pass.

```
// From DarshanBookingController.java
@PostMapping("/book")
public ResponseEntity bookSlot(@RequestBody DarshanBooking booking) {
    // 1. Generate unique QR Pass entry code
    String checkInId = "DS-" + UUID.randomUUID().toString().substring(0, 6).toUpperCase();
    booking.setId(checkInId);
    booking.setStatus("CONFIRMED");

    // 2. Save reservation to MongoDB
    DarshanBooking saved = darshanRepository.save(booking);

    // 3. Dispatch entry pass email with QR Link
    try {
        emailService.sendDarshanPassEmail(
            booking.getEmail(), booking.getDevoteeName(), checkInId,
            booking.getBookingDate(), booking.getSlotTime(), booking.getGotra(),
            booking.getGroupSize()
        );
    } catch (Exception e) {
        System.err.println("Mail dispatch failed: " + e.getMessage());
    }
    return ResponseEntity.ok(saved);
}
```

5.1 Core Java API utilities utilized in code:

- **java.util.UUID:** Generates unique IDs to prevent duplicate booking and transaction numbers.
- **BCryptPasswordEncoder:** A cryptographic hash function mapping passwords into a 60-character bcrypt hash before database storage.
- **SimpleDateFormat:** Translates frontend ISO dates into clean formatted receipt values.
- **Java MimeMessageHelper:** Encapsulates multi-part HTML parsing, formatting text styles, margins, and gradients in the transactional receipt email template.